

TOPIC TWO: Software Reliability

Introduction to Software Reliability

Software reliability is defined as the probability that a software system will perform its required functions correctly without failure for a specified period under stated conditions. It forms a cornerstone of software quality assurance, guaranteeing that applications deliver consistent performance, maintain user trust, and avoid disruptions during normal operation. Unlike hardware reliability (affected by physical wear), software reliability depends entirely on code quality, testing thoroughness, and design decisions, bugs don't "wear out" but remain until discovered and fixed. Critical for mission-essential systems like banking applications, medical devices, and university portals, high reliability prevents financial loss, health risks, and operational chaos.

Software Reliability Models

Software reliability models are mathematical frameworks that predict failure behavior, estimate remaining defects, and determine optimal release timing based on observed testing data. These models enable data-driven decisions, transforming subjective "feels ready" judgments into objective "99% confident" release criteria essential for mission-critical systems.

a) Jelinski-Moranda Model

The Jelinski-Moranda Model measures software reliability based on remaining faults and predicts Mean Time Between Failures (MTBF), the average time before the next crash. Each time a crash is fixed, the model assumes you've removed one fault, so MTBF increases as fewer faults remain.

For example, A registration system crashes 15 times in testing. After repairing those 15 faults, developers calculate expected hours to the next failure during heavy exam signup periods, instead of only noting total crash count.

Early tests uncover simple bugs, with failures occurring every 10 hours. Later tests reveal complex faults, extending intervals to 100 hours between crashes. The model charts this rising reliability to determine testing endpoint before deployment.

Each repair boosts MTBF. Developers confirm intervals averaging 75 hours between problems, signaling readiness for release. A portal begins with about 30 faults. After correcting 20 faults across 5,000 test hours, MTBF rises from 50 hours to 250 hours, projecting over 200 hours of stable operation during peak registration before another failure surfaces. Plan server upgrades in advance.

b) Musa-Okumoto Model

The Musa-Okumoto Model predicts the **number of crashes** expected during future usage time (actual hours people spend using the app), rather than calendar days.

In always-running systems like WhatsApp or university learning portals, teams monitor total failures after cumulative usage reaches 1 million hours across all user sessions, instead of tracking days since the last update.

Light morning tests often miss bugs that surface only during heavy evening usage across time zones. The model forecasts failures during peak loads, like 10,000 students accessing exam registration simultaneously, determining precise maintenance timing by calculating expected failures per additional usage hour, so teams schedule updates before traffic spikes cause outages.

c) Goel-Okumoto Model

Goel-Okumoto estimates **number of bugs still hiding** in the system using Non-Homogeneous Poisson Process (NHPP). It predicts total faults (a) and subtracts detected faults to show remaining ones, with detection slowing as easy bugs get removed first.

University exam portal fails 45 times after 10,000 test hours. Model estimates **60 total faults**, so **15 bugs still hiding**—projecting when those will surface instead of assuming infinite problems. Early testing uncovers obvious faults quickly. Harder faults take longer to find, following S-shaped curve where cumulative failures approach maximum. Model calculates **remaining faults = total faults - detected faults** for release decisions.

Developers track when observed failures near predicted total. **15 faults remaining** signals when additional testing yields minimal gains. Portal shows 60 total faults. After detecting 45, **15 bugs still hiding**—forecast stable operation after 5,000 more hours to find them. Schedule final validation before exams.

Factors Affecting Software Reliability

Several factors impact how reliable a software system is:

Software Complexity

Software complexity arises when applications grow by adding multiple interconnected features. Each new capability—like effects, streaming, or recommendations—creates dependencies across code sections. These interactions multiply, so a change in one area unexpectedly disrupts others, increasing failure likelihood as the system becomes harder to fully understand and control.

Development Methodology

Development methodology refers to the structured process teams follow to build software. Rushed approaches skip critical steps like planning, code reviews, and incremental testing, allowing defects to embed deeply. Methodical processes—planning features first, reviewing code collaboratively, testing components separately before integration—detect issues early, ensuring stability from initial deployment.

Testing & Debugging

Testing verifies application functionality under controlled conditions, while debugging isolates and corrects root causes of failures. Without thorough testing across device types, networks, and usage scenarios, defects remain hidden until production. Comprehensive testing simulates real conditions to expose problems, followed by targeted debugging to repair specific code faults, preventing user-facing disruptions.

Environmental Conditions

Environmental conditions encompass external factors like hardware, networks, and operating systems that affect software performance. The same application runs smoothly on high-end devices with fast connections but fails on budget phones, slower networks, or older software versions due to resource constraints or compatibility gaps. Accounting for this variability during design and testing ensures consistent operation across diverse user environments.

Techniques to Improve Software Reliability

Organizations use various strategies to enhance reliability:

Automated Testing

Automated testing involves using software tools, often called scripts or bots, to systematically execute predefined tests on an application. These tools mimic user interactions, like entering data or navigating interfaces, across various simulated environments. The process runs repeatedly, especially after code changes, to verify that features work correctly without human intervention each time. It detects defects early by comparing actual outputs against expected results, flags inconsistencies, and generates detailed reports for developers. This technique scales efficiently, handling thousands of tests in minutes, which manual testing couldn't match.

Key benefits include faster feedback loops in development, higher test coverage for complex scenarios, and reduced human error. It ensures consistency since tests follow exact scripts every run. However, it requires upfront investment in creating reliable test scripts and maintaining them as the software evolves.

Redundant Systems

Redundant systems deploy duplicate hardware, software, or data copies across multiple locations to prevent single points of failure. If the primary component fails—due to hardware issues, power outages, or overload—the backup seamlessly activates, maintaining uninterrupted service. Replication happens in real-time, often using synchronization protocols to keep copies identical. Failover mechanisms monitor health metrics like response times and switch automatically within milliseconds.

This approach boosts availability by distributing risk geographically and diversifying infrastructure. It minimizes downtime, preserves user sessions, and supports recovery without data loss. Drawbacks include higher costs for extra resources and complexity in managing synchronized states.

Regular Updates & Patches

Regular updates and patches deliver targeted fixes, enhancements, or security reinforcements to software in controlled releases. Developers identify issues through monitoring, testing, or user reports, then isolate and resolve them without altering the entire system. These roll out incrementally, often overnight or via background downloads, to billions of devices, with mechanisms like staged rollouts testing on small user groups first.

The technique prioritizes minimal disruption, using versioning to ensure backward compatibility. It addresses vulnerabilities promptly, improves performance, and incorporates user-requested features. Challenges involve coordinating global deployments and verifying patches don't introduce new bugs.

Load Balancing & Fault Tolerance

Load balancing distributes incoming user requests across multiple servers or resources using algorithms that monitor capacity, response times, and traffic patterns. It prevents overload by routing traffic evenly, scaling dynamically as demand surges. Fault tolerance complements this by detecting failures, via heartbeats or probes, and rerouting affected traffic instantly, often with self-healing capabilities like automatic restarts.

Together, they ensure scalability for massive concurrency and resilience against partial outages. Health checks continuously validate server status, enabling proactive adjustments. This maintains consistent performance but demands sophisticated orchestration tools and can add latency if not optimized.

Combined Impact on Software Reliability

These techniques interlock to create robust systems: automated testing catches issues pre-release, redundant systems provide instant backups, regular patches heal live problems, and

load balancing with fault tolerance handles scale and failures. The result is near-continuous uptime—often measured in "nines" like 99.999%—reducing crashes, buffering, or complaints during peak usage. In high-stakes apps, this reliability builds user trust and supports growth across diverse networks.

Software Reliability Measures

Reliability is assessed using key performance metrics:

a) Mean Time Between Failures (MTBF)

Calculates the average duration between one failure and the next during operation. Higher MTBF signals greater reliability, as the system operates longer between breakdowns. For example, an online banking app with an MTBF of 1,000 hours encounters issues far less often than one at 200 hours, ensuring users trust consistent performance.

MTBF FORMULA=Total Operational Time/ Number of Failures

Example One

As a Software Testing student, you manage testing for a new mobile banking app used by 50,000 users. Over a 30-day beta test (720 total hours of operation across all devices), the app experienced 12 crashes due to network glitches on MTN 4G. Calculate the MTBF and recommend if it's ready for production (target: >500 hours).

Solution:

Total Operational Time = 720 hours

Number of Failures = 12

MTBF=720/12=60 hours

At 60 hours, it's below target, add automated testing on slow networks to boost MTBF before launch.

Example Two

MUBSEP ran for 2,000 hours during end-of-semester exams. Logs show 5 major failures which affected the exam process. Compute MTBF.

Solution:

Total Operational Time = 2,000 hours

Number of Failures = 5

MTBF=2,000/5=400 hours

Worse than competitor's 800 hours, deploy redundant login servers to double effective MTBF via failover.

Example Three

Wicky Beddings website operated 1,500 hours during a black November sale. It had 3 server crashes from user overload. As a software Testing students, you have been asked to resolve the failure. Calculate the MTBF.

Solution:

Total Operational Time = 1,500 hours

Number of Failures = 3

MTBF=1,500/3=500 hours

Example Four

Your QA team tests a new automation tool over 500 hours, logging 2 false positives (buggy detections). Calculate MTBF.

Solution:

Total Operational Time = 500 hours

Number of Failures = 2

MTBF=500/2=250 hours

b) Mean Time To Failure (MTTF)

Used for non-repairable systems. Measures how long software operates before its first failure. Example: A one-time-use embedded system in spacecraft should have a high MTTF to ensure mission success.

MTBF Formula: $MTTF = \text{Total Operational Time} \div \text{Number of Units Tested}$

MTTF measures time to first failure in non-repairable systems.

Example One

A delivery company tests 15 single-use drone navigation firmware updates over 120-hour simulated flights. Four drones crash from GPS errors with total uptime of 1,500 hours.

Calculate MTTF compared to old version's 90 hours.

Solution:

Total Operational Time = 1,500 hours

Number of Units Tested = 15

MTTF = $1,500 \div 15 = 100$ hours

Better than previous but needs GPS testing improvement to reach 150 hours target.

Example Two

A company certifies 25 non-repairable fire alarm sensors for 400-hour testing cycle. Three sensors fail from heat drift with total uptime of 9,500 hours. Safety standard requires 350 hours MTTF.

Solution:

Total Operational Time = 9,500 hours

Number of Units Tested = 25

MTTF = $9,500 \div 25 = 380$ hours

Passes certification. Add extreme temperature testing for reliability.

Example Three

An election app tests 40 single-use ticket scanner modules over 12-hour event simulation. Six modules fail from reader errors with total uptime of 400 hours. Event needs 8 hours MTTF minimum.

Solution:

Total Operational Time = 400 hours

Number of Units Tested = 40

MTTF = $400 \div 40 = 10$ hours

Exceeds requirement. Improve scanner calibration to prevent reader failures.

c) Mean Time To Repair (MTTR)

Indicates how long it takes to fix a failure and restore service. Lower MTTR means quicker recovery.

Example: A social media app like Instagram aims for a low MTTR to minimize downtime during outages.

MTTR Formula: $MTTR = \text{Total Repair Time} \div \text{Number of Failures}$

Example One

School Pay, experiences 8 login crashes during peak evening usage. The support team logs total repair time as 120 minutes across all fixes. Calculate MTTR. Target is under 20 minutes per incident for user retention.

Solution:

Total Repair Time = 120 minutes

Number of Failures = 8

MTTR = $120 \div 8 = 15$ minutes

Meets target. Automated rollback scripts helped achieve fast recovery.

Example Two

Jumia has 5 payment processing failures during Black Friday sales. Developers spend 90 minutes total diagnosing and patching the gateway integration. Calculate MTTR vs. last quarter's 25 minutes.

Solution:

Total Repair Time = 90 minutes

Number of Failures = 5

MTTR = $90 \div 5 = 18$ minutes

Improved from previous. Add payment gateway monitoring alerts for even faster detection.

Example Three

Pearl Stream Uganda gets 12 buffering failures on MTN 4G networks. Engineering team records 180 minutes total fix time including code deploy. Target MTTR is 12 minutes for smooth viral content.

Solution:

Total Repair Time = 180 minutes

Number of Failures = 12

MTTR = $180 \div 12 = 15$ minutes

Slightly above target. Implement pre-staged fixes to reduce deployment time.

Example Four

During semester exams, a student banking app has 6 two-factor authentication failures. IT support spends 72 minutes total across hotfixes and database resets. Bank policy requires MTTR under 15 minutes.

Solution:

Total Repair Time = 72 minutes

Number of Failures = 6

MTTR = $72 \div 6 = 12$ minutes

Under target. Dashboard monitoring enabled quick root cause identification.

Example Five

Safe Car faces 10 GPS location failures during rush hour. The fix team logs 200 minutes total for server restarts and API updates. Service goal is MTTR below 25 minutes to keep drivers active.

Solution:

Total Repair Time = 200 minutes

Number of Failures = 10

MTTR = $200 \div 10 = 20$ minutes

Acceptable but room for improvement. Add automated GPS failover to cut diagnosis time.

d) Failure Rate

Failure rate in software reliability measures the frequency at which the software fails to perform as expected during actual operation or production use. It quantifies real-world non-performances, such as crashes, outages, or incorrect behaviors, relative to total usage opportunities or time periods. This metric helps assess overall system stability post-deployment. Teams use it to gauge reliability and prioritize improvements in live environments. A lower failure rate means more stable software.

Failure Rate Formula: $\text{Failure Rate} = \text{Number of Failures} \div \text{Total Operational Time}$

Example One

Achi clothing Store, an online store, operates 300 hours during holiday sales. Checkout crashes occur 18 times from payment bugs. Calculate failure rate compared to regular season's 0.04 per hour.

Solution:

Number of Failures = 18

Total Operational Time = 300 hours

Failure Rate = $18 \div 300 = 0.06$ failures per hour

Higher than normal. Implement load testing before peak sales periods.

Example Two

Netflix streams for 1,200 hours across Uganda on MTN 4G. Video buffering failures happen 72 times. Target failure rate is 0.05 or lower per hour for viral content.

Solution:

Number of Failures = 72

Total Operational Time = 1,200 hours

Failure Rate = $72 \div 1,200 = 0.06$ failures per hour

Slightly above target. Optimize for slow networks with adaptive bitrate.

Example Three

Stanbic mobile banking app runs continuously for 2,000 hours monthly. Login timeouts affect users 40 times. Bank standard requires failure rate below 0.02 per hour.

Solution:

Number of Failures = 40

Total Operational Time = 2,000 hours

Failure Rate = $40 \div 2,000 = 0.02$ failures per hour

Meets standard. Server monitoring prevents future spikes.

Example Four

Uber operates 800 hours during rush hour peaks. Map loading fails 48 times from GPS issues. Service goal is failure rate under 0.06 per hour.

Solution:

Number of Failures = 48

Total Operational Time = 800 hours

Failure Rate = $48 \div 800 = 0.06$ failures per hour

At target limit. Add offline map caching for better stability.

Assignment: Calculate Reliability Metrics

Assignment submission deadline: 8th March 2026

Email: testingsoftware62@gmail.com

MTBF Questions

1. The MUBS student portal runs for 1,800 hours during registration week and records 6 login failures. Compute MTBF to assess readiness for peak traffic.
2. A lecture scheduling app operates 2,200 hours over a semester with 11 database timeouts. Determine MTBF for maintenance planning.
3. The fee payment gateway accumulates 1,500 hours of use with 5 transaction errors. Calculate MTBF to decide on server upgrades.

MTTR Questions

1. IT team fixes 5 exam system outages in times: 1.2 hours, 2.8 hours, 1.5 hours, 3.0 hours, 2.2 hours. Compute average MTTR for efficiency audit.
2. Debugging the gradebook tool takes 1.8, 2.5, 3.2, 1.9, and 2.1 hours for 5 issues. Find MTTR to evaluate response protocols.
3. Course registration portal has 4 repairs logged at 0.8, 1.5, 2.3, and 1.7 hours. Calculate MTTR for team training needs.

Failure Rate Questions

1. Library booking system runs 6,000 hours with 24 access denials. Compute failure rate per hour to review access controls.
2. Online quiz platform tests for 4,500 hours yielding 15 submission glitches. Determine failure rate for patch prioritization.
3. Campus email notifier operates 3,200 hours with 10 delivery failures. Calculate failure rate to plan redundancy.

MTTF Questions

1. Three prototype enrollment apps fail after 2,800, 3,100, and 2,500 hours. Compute MTTF for beta extension.
2. Four versions of a timetable generator crash at 1,900, 2,200, 1,700, and 2,000 hours. Find MTTF for redesign.
3. Lab simulation software ends after 4,000, 4,500, and 3,800 hours each. Calculate MTTF for resource allocation.